

系统时间、时区、RTC 与 chrony

从“现在几点”拆出显示、硬件基准、时间源选择与本机跟踪：用证据证明时间链真正成立。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：外部时间源只有经过解析、NTP 往返、样本积累、来源选择和本机跟踪，才可能真正控制 system clock；时区和 RTC 又分别处在显示层与启动基准层。

专题地图

- 知识专题 一个“时间”为什么要拆成 UTC、本地时间、系统时钟与 RTC
- 操作专题 用 `date` 与 `timedatectl` 建立可比较的时间基线
- 操作专题 查询 RTC，并控制 system clock 与 RTC 的复制方向
- 知识专题 chronyd 从“知道一个服务器”到“控制系统时钟”经历哪些状态
- 操作专题 配置 `server`、`pool`、`iburst` 与持久校时策略
- 操作专题 解读 `chronyc activity`、`sources -v` 与 `tracking`
- 诊断专题 `chronyd active`，但指定来源持续显示 `^?`
- 诊断专题 大偏差时怎样选择 `slew`、`step` 与 `makestep`
- 经典任务 配置指定时区和时间源，并恢复有效跟踪
- 参考解答 从基线、最小修改到分层验收
- 本章收束 把时间管理变成证据矩阵

阅读时持续回答

1. 当前问题发生在显示层、system clock、RTC，还是时间同步链？
2. `active`、`enabled`、来源可用、来源被选中和本机已同步分别由什么证据证明？
3. `sources` 与 `tracking` 各自回答哪个问题？
4. `^?` 的下一条最有区分度的证据是什么？
5. 当前偏差应继续 `slew`，还是确有授权执行 `step`？
6. 这条验证能证明什么，又不能证明什么？

01分清时间对象UTC / local / system clock / RTC

02建立基线date / timedatectl / hwclock

03核对配置server / pool / iburst / makestep

04观察来源activity / sources -v

05判断跟踪tracking / offset / leap status

06诊断与控制风险DNS / UDP 123 / slew / step

本章只轻量引用第 13 章的日志证据、第 15 章的计划任务影响，以及网络、DNS 与防火墙章节的接口；不展开 PTP。

第 14 章 · 正文

一台 Linux 主机上同时存在多种“时间事实”。`date` 展示的是 system clock 经时区规则转换后的结果；RTC 在主机关闭时保留启动基准；`chronyd.service` 只描述守护进程是否在运行；`chronyc sources -v` 描述时间源的测量与选择；`chronyc tracking` 才进一步说明本机时钟正在怎样跟随参考时间。把这些对象混为一谈，就会出现最典型的误判：服务明明是 active，却仍然没有有效参考源；时区差八小时，却误以为必须强制校时；来源名称已经出现，就提前宣布同步成功。

本章沿着“对象 → 状态 → 查询 → 配置 → 来源证据 → 本机跟踪 → 诊断与风险控制”推进。前一章负责日志检索，本章只说明时间错误会怎样削弱日志证据；下一章负责计划任务，本章只建立时区变更和时间跳变对日历调度的接口边界。

概念

系统时钟 (system clock) 是内核维护、运行中程序通常读取的墙上时钟。文件时间戳、日志记录、证书有效期和许多认证判断依赖它。它可以由管理员手工修改，也可以由 `chronyd` 通过改变走速或直接跳变来校正；因此“系统时钟当前可读”不等于“它来自可信来源”，更不等于“校正过程对业务无风险”。

概念

UTC、本地时间与时区 描述的是同一时刻的不同解释层。UTC 提供跨主机比较的共同基准；时区规则把 UTC 转换为本地显示和日历语义。修改时区通常不会平移绝对时刻，却会改变用户看到的时间以及按本地日历解释的任务。显示差八小时可能只是时区错误，不能直接推出 system clock 偏差八小时。

概念

RTC (Real-Time Clock) 是硬件或虚拟硬件提供的持久时钟，主要在关机期间继续计时，并在启动阶段为 system clock 提供初始基准。运行中应用通常不直接依赖 RTC。`hwclock --systohc` 与 `--hctosys` 的复制方向相反，任一方向都可能把错误值扩散到另一个对象，因此操作前必须先明确谁是真源、谁是目标。

概念

NTP 同步状态 不是单一布尔值，而是一条状态链：服务正在运行、来源名称可解析、NTP 报文可往返、样本足够、来源可选、来源被选中、本机进入有效跟踪。`systemctl is-active chronyd` 只覆盖第一层；即使 `timedatectl` 显示 NTP service active，也不能替代 `sources` 与 `tracking` 的证据。

概念

时间源 (time source) 是 `chronyd` 用来测量参考时间的服务器、对等体或参考时钟。配置文件中出现一个名称，只说明“声明过这个对象”；它还要经过名称解析、网络往返、时间质量检查与来源选择。`sources -v` 中的模式字符说明来源类型，选择字符则说明它当前能否参与同步以及是否被选中。

概念

offset 表示本地时钟与参考时间之间估计的差异，但不同输出中的 offset 语义并不完全相同：`sources` 面向某个来源的最近测量，`tracking` 面向本机时钟相对当前参考时间的剩余校正与统计。offset 越小通常越好，却不不存在适用于所有系统的固定毫秒阈值；验收必须结合任务、网络延迟和业务容忍度。

概念

stratum 表示时间源距离参考时钟的层级，而不是“数值越小就绝对越准确”的排行榜。一个网络不稳定的低层级来源可能不如稳定的高一层来源。判断来源时应同时观察可达性、测量误差、选择状态和本机 tracking，

不能只盯住 Stratum 一列。

概念

slew 与 step 是两种不同的校正方式。slew 通过暂时加快或减慢时钟走速逐步消除偏差，时间连续但大偏差收敛较慢；step 直接把墙上时钟跳到新值，修正迅速却可能让日志顺序、缓存过期、认证、数据库事务和调度行为发生异常。`makestep` 是对这种风险的有限授权，不是看到 `^?` 后的通用修复命令。

操作语义 以下入口分别观察显示层、系统配置、RTC、来源状态和本机跟踪。先理解命令作用对象，再记关键形式。

date

SYNOPSIS

```
date [OPTION]... [+FORMAT]
date [-u] [+FORMAT]
```

读取 system clock，并按当前时区或 UTC 格式化输出。它适合建立本地时间与 UTC 的并列基线，但不能证明时间源有效。

重要参数 / 形式

```
date '+%F %T %Z %Z'
```

显示日期、时间、数值偏移和时区缩写。

```
date -u '+%F %T %Z %Z'
```

按 UTC 显示同一 system clock。

```
date --iso-8601=seconds
```

生成便于记录和交换的 ISO 8601 时间。

timedatectl

SYNOPSIS

```
timedatectl [OPTIONS...] COMMAND ...
```

统一查询和修改 system clock、时区、RTC 解释模式及网络时间同步开关。综合状态是入口，不是 chrony 来源与跟踪的终局证据。

重要参数 / 形式

```
timedatectl
```

查看 Local time、Universal time、RTC、Time zone 和同步状态。

```
timedatectl list-timezones
```

列出可用时区名称。

```
timedatectl set-timezone Asia/Shanghai
```

持久设置时区。

```
timedatectl set-ntp true
```

请求启用系统识别的网络时间同步服务。

hwclock

SYNOPSIS

```
hwclock --show [--utc|--localtime]
hwclock --systohc
hwclock --hctosys
```

直接观察或复制 RTC 与 system clock。名称相似但方向相反；除只读查询外，都应先明确真源和目标。

重要参数 / 形式

--show

只读显示 RTC。

--systohc

把 system clock 写入 RTC。

--hctosys

用 RTC 设置 system clock，运行中使用可能造成时间跳变。

chronyc activity / chronyc sources -v

SYNOPSIS

```
chronyc activity
chronyc sources [-a] [-v]
```

activity 汇总来源在线、离线和未解析状态；**sources -v** 展示每个来源的模式、选择状态、可达性、最近接收和最近样本。它们回答“来源怎样”，不直接等价于“本机时钟怎样”。

重要参数 / 形式

^*

server 来源，且当前是最佳同步来源。

^+

server 来源，参与组合但不是最佳来源。

^?

当前不可选，可能是不可达、未同步或样本不足。

Reach / LastRx / Last sample

分别观察近期响应历史、最近接收时间和最近偏差估计。

chronyc tracking

SYNOPSIS

```
chronyc tracking
```

展示 chronyd 当前如何控制本机 system clock。它回答参考来源、剩余校正、频率估计、误差边界和 leap 状态，是同步验收的核心证据之一。

重要字段

Reference ID / Ref time

当前参考对象和最近一次参考更新时间。

System time / Last offset / RMS offset

本机剩余校正、最近偏差和偏差统计。

Frequency / Residual freq / Skew

本机振荡器频率估计及其不确定性。

Leap status

Normal 等状态可参与正常验收；Not synchronised 表示尚未有效同步。

/etc/chrony.conf

SYNOPSIS

```
server hostname [option ...]
pool hostname [option ...]
makestep threshold limit
```

这是 RHEL 9 中 chronyd 的主要持久配置入口。编辑时应保留既有有效指令、避免重复来源，并通过服务重启和运行时证据确认新配置真正生效。

重要参数 / 形式

server time.example.com iburst

配置一个 NTP 服务器，并在初始阶段加快获得首批有效测量。

```
pool pool.example.com iburst
```

从一个池名称管理多个可替换来源。

```
iburst
```

加快首次更新，不表示长期高频轮询。

```
makestep 1.0 3
```

在有限的前几次更新中，允许超过阈值的偏差被 step。

chronyc makestep

SYNOPSIS

```
chronyc makestep
chronyc makestep <threshold> <limit>
```

控制正在运行的 chronyd 立即或临时允许 step。它可能造成墙上时钟跳变，不会修复 DNS、UDP 123 或错误时间源。

重要参数 / 形式

```
chronyc makestep
```

取消剩余 slew，并按当前估计立即跳变。

```
threshold
```

触发自动 step 的偏差阈值，单位为秒。

```
limit
```

该临时规则允许作用的后续更新次数。

[知识专题] 一个“时间”为什么要拆成 UTC、本地时间、系统时钟与 RTC

排查时间问题的第一步不是立即运行 `chronyc makestep`，而是确认到底哪个对象不符合预期。显示差八小时可能只是时区错误；系统刚启动时 RTC 偏差可能导致初始系统时间错误；chronyd 正常运行却没有参考源，则属于同步链而不是显示层。

① [知识点] UTC 表示时刻，本地时间表示解释结果

UTC 是跨主机比较时间的稳定基准。时区数据库规定某个地区相对 UTC 的偏移、历史变更以及夏令时规则。本地时间由同一 UTC 时刻按当前时区转换而来。

```
date                                # 按当前时区显示
date -u                             # 按 UTC 显示
date '+%F %T %z %Z'                # 日期、时间、数值偏移、时区缩写
date -u '+%F %T %z %Z'
```


若 `date` 与 `date -u` 的差值符合时区偏移，不能仅凭两者不同判定系统时钟错误。修改时区会改变本地显示以及按本地日历解释的任务，但不会把同一时刻的 UTC 值平移。

② [知识点] system clock 是运行中系统的主要墙上时钟

系统时钟由内核维护。应用读取当前日期和时间、文件系统写入时间戳、日志记录事件、TLS 检查证书有效期时，通常依赖这个时钟。它可以被手工设置，也可以由 `chronyd` 通过渐进校正或跳变校正调整。

系统时钟可能向前或向后跳变，因此“经历了多长时间”的测量通常更适合单调时钟。单调时钟不在本章深入，但应记住：墙上时钟适合回答“事件发生在什么时刻”，单调时钟适合回答“经过了多久”。

③ [知识点] RTC 为关机期间提供基准，不等于 system clock

RTC 可以在主机关机后继续计时。系统启动时，内核或早期用户空间可从 RTC 初始化系统时钟；运行期间通常由系统时钟服务应用。两者能够互相复制，但复制方向必须明确：

```
RTC → system clock: hwclock --hctosys
system clock → RTC: hwclock --systemhctosys
```

这两个操作会改变目标对象。没有调查当前值、方向和业务影响时，不应把它们当作普通“同步命令”随意执行。

④ [知识点] 服务器 RTC 通常保持 UTC

RTC 本身通常不保存时区信息，系统必须约定其读数按 UTC 还是本地时间解释。服务器一般保持 UTC，以减少夏令时切换和多操作系统解释差异。`timedatectl set-local-rtc 1` 只适合确有兼容需求的环境，并会改变 `/etc/adjtime` 中的模式记录。

⑤ [知识点] 时钟链存在多个独立状态

```
时区正确
≠ 系统时钟正确
≠ RTC 正确
≠ chronyd 正在运行
≠ 已取得有效 NTP 测量
≠ 已选中参考源
≠ 系统已进入有效跟踪
```

每个等号右侧都需要独立证据。任何一个局部状态都不能替代整条时间链。

Cheatsheet

UTC 是时刻基准；时区负责显示和日历解释；system clock 是运行中程序使用的墙上时钟；RTC 为关机期间保存基准；改时区不等于校时。

[操作专题] 用 `date` 与 `timedatectl` 建立可比较的时间基线

时间变更前要先保存基线。基线至少同时包括本地时间、UTC、时区、RTC、网络同步服务和系统同步判断。只截取一行“Local time”会丢失关键状态。

① [操作] 同时记录本地时间与 UTC

作用对象：系统时钟的两种显示方式。基本语义：`date` 读取系统时钟；`-u` 强制按 UTC 显示。典型形式：

```
date '+local=%F %T %z %Z'
date -u '+utc=%F %T %z %Z'
```

验证与边界：两条命令几乎同时执行但仍可能有少量秒差；比较重点是时区偏移和时刻逻辑，不是要求秒字段完全相同。

② [操作] 逐行解读 `timedatectl`

```
timedatectl
```

常见字段回答不同问题：

字段	回答的问题	不能替代的证据
Local time	当前时区下显示什么时间	不能证明 UTC 正确
Universal time	当前 UTC 时刻	不能证明来源可达
RTC time	RTC 当前读数	不能证明运行中应用使用它
Time zone	当前时区和偏移	不能证明系统已同步
System clock synchronized	systemd 视角的同步判断	不替代 <code>sources</code> 与 <code>tracking</code>
NTP service	网络时间同步服务是否启用/活动	不代表已有有效参考源
RTC in local TZ	RTC 是否按本地时间解释	不代表 RTC 数值正确

③ [操作] 查询有效时区并设置目标时区

作用对象：系统的时区链接和解释规则。基本语义：`set-timezone` 选择时区数据库中的一个有效名称。典型形式：

```
timedatectl list-timezones | grep '^Asia/'
timedatectl set-timezone Asia/Shanghai
```

验证：

```
timedatectl
date '+%F %T %z %Z'
date -u '+%F %T %z %Z'
readlink -f /etc/localtime
```

边界： `/etc/localtime` 的链接是持久配置证据； `date` 是当前显示证据。修改时区后，要在第 15 章的计划任务范围中重新核对使用本地日历的 `cron` 与 `OnCalendar=`。

④ [操作] 区分手工设置时间与启用网络同步

```
timedatectl set-time '2026-07-12 15:30:00'
timedatectl set-ntp true
```

`set-time` 直接改变系统时钟；系统正在使用网络时间同步时，通常应先明确停用该同步机制，否则手工设时可能被拒绝或很快被重新校正。`set-ntp true` 请求启用系统识别的网络时间同步服务；在 RHEL 9 中通常由 `chronyd` 承担这一职责。启用同步后，不应再把手工设置时间作为常规维护路径。

安全边界：手工改时可能导致日志倒序、认证失败、证书判断异常和计划任务行为变化。考试题目未明确要求时，不应为了让输出“看起来正确”而手工设置时间。

⑤ [操作] 记录基线而不伪造真实输出

推荐把命令和真实输出保存在练习记录中：

```
{
  echo '=== local ==='
  date '+%F %T %z %Z'
  echo '=== utc ==='
  date -u '+%F %T %z %Z'
  echo '=== timedatectl ==='
  timedatectl
} | tee /root/time-baseline.txt
```

本讲义不提供虚构的 IP、offset、Reference ID 或 Reach 值。真实环境中必须读取本机证据。

Cheatsheet

先记录 ``date``、``date -u``、``timedatectl``；列时区用 ``list-timezones``，设置用 ``set-timezone``；改时区后验证 ``/etc/localtime`` 和本地/UTC 显示。

[操作专题] 查询 RTC，并控制 system clock 与 RTC 的复制方向

RTC 操作容易因为名称相似而方向相反。操作前先写清楚“以谁为真源、要改变谁”。

① [操作] 只读查询 RTC

```
hwclock --show
hwclock --show --utc
```

`hwclock --show` 读取硬件时钟并按系统记录的 RTC 模式解释。`--utc` 或 `--localtime` 可以显式指定解释方式，但不应在不了解当前 `/etc/adjtime` 时混用并据此修改系统。

② [操作] 查看 RTC 模式记录

```
timedatectl
cat /etc/adjtime
```

`/etc/adjtime` 的第三行通常记录 `UTC` 或 `LOCAL`。该文件还可保存 RTC 漂移相关信息。只需要判断模式时，优先读取，不要先执行会写入它的命令。

③ [操作] 从 system clock 写入 RTC

```
hwclock --systohc
```

作用对象：RTC。前提：已确认 system clock 是可信真源，且需要让 RTC 保存该基准。验证：再次读取 `hwclock --show`，并确认模式仍符合预期。边界：命令成功只证明复制完成，不证明 system clock 原本正确。

④ [操作] 从 RTC 初始化 system clock

```
hwclock --hctosys
```

作用对象：system clock。风险：运行中执行会使系统时间跳变，可能破坏日志、认证和应用顺序。常规启动流程会处理 RTC 初始化，日常排障不应把它作为第一动作。

⑤ [边界] 虚拟机中的 RTC 可能不是独立物理设备

云平台或虚拟机可能提供虚拟 RTC，也可能由宿主机、虚拟化工具或启动机制影响客户机时间。`hwclock` 失败或 RTC 与宿主机相关，并不直接证明 `chronyd` 配置错误。此类平台特性需要真实环境验证。

Cheatsheet

`--show` 只读 RTC；`--systohc` 是系统时钟写 RTC；`--hctosys` 是 RTC 写系统时钟；操作前先明确真源和目标。

[知识专题] chronyd 从“知道一个服务器”到“控制系统时钟”

经历哪些状态

chronyd 的工作不是收到一个服务器地址后立即把本机时间复制过去。它需要解析名称、交换 NTP 报文、积累测量、排除异常来源、选择参考源，然后根据偏差和策略校正系统时钟。

① [知识点] 配置状态只是声明

`/etc/chrony.conf` 中出现：

```
server time.example.com iburst
```

只说明管理员声明了一个来源。它不能证明：

- 名称能够解析；
- 路由存在；
- UDP 123 请求和响应能够往返；
- 对端提供有效 NTP 响应；
- 已取得足够样本；
- 该源通过选择算法；
- 本机已经跟踪它。

② [知识点] 服务状态与同步状态分离

```
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
```

`active` 回答守护进程当前是否运行；`enabled` 回答重启后的持久启动配置。两者都不能证明源可用或系统已同步。

③ [知识点] 来源测量、选择和跟踪是三层证据

<code>chronyc activity</code>	→ 已知来源的在线、离线、未解析数量
<code>chronyc sources -v</code>	→ 每个来源的模式、可达性、样本和选择状态
<code>chronyc tracking</code>	→ 本机相对选中参考时间的跟踪状态

最小闭环要求：服务存在、来源可用、至少一个有效参考源被选择、tracking 不处于未同步状态。

④ [知识点] stratum 是到参考时钟的层级距离

stratum 反映来源与参考时钟之间的层级。直接连接参考时钟的服务器通常是较低 stratum，沿同步链向下增加。它不是“数值越小就一定越准确”的简单排名；网络抖动、源质量、根距离和选择算法同样重要。

⑤ [知识点] offset 不是单一固定验收阈值

`chronyc sources -v` 和 `chronyc tracking` 中有多个偏差指标。它们表达的对象和时间窗口不同。不能规定所有主机都必须小于同一个毫秒值。考试最小判断通常是进入有效同步；生产环境还要按 Kerberos、证书、数据库和分布式系统要求设定偏差目标。

Cheatsheet

配置只声明来源；`active/enabled` 只证明服务层；`activity` 看来源解析状态；`sources` 看测量与选择；`tracking` 看本机时钟。

[操作专题] 配置 `server`、`pool`、`iburst` 与持久校时策略

编辑 `chrony` 配置时应在现有系统上做最小修改。考试和真实服务器都可能已经包含有效的 `driftfile`、`makestep`、`rtcsync` 或其他来源。整文件覆盖会删除无关但必要的状态。

① [操作] 变更前检查配置和备份

```
grep -Ev '^s*(#|$)' /etc/chrony.conf
cp -a /etc/chrony.conf /etc/chrony.conf.pre-rhcsa14
```

备份名要避免被配置加载机制误当成有效片段。RHEL 9 的确定主入口是 `/etc/chrony.conf`；是否还加载 `drop-in`，应依据本机文件内容和包版本确认，不跨发行版假设。

② [配置] `server` 指定一个来源

```
server time.example.com iburst
```

`server` 指定一个 NTP 来源的名称或地址。一个名称若解析为多个地址，`chronyd` 的具体地址管理仍由实现处理；在学习模型中，`server` 表示一个配置来源条目。

③ [配置] `pool` 从一个池名称管理多个来源

```
pool pool.example.com iburst
```

`pool` 适合一个 DNS 名称背后提供多个可替换服务器的场景。它与“配置多个独立 `server` 行”都能增加来源冗余，但来源管理语义不同。

④ [参数] `iburst` 加快首次有效测量

`iburst` 在来源初次可用或尚未同步时快速发送一组请求，以更快取得初始测量。它不是让 `chronyd` 永久高频轮询，也不能绕过 DNS、路由、防火墙和远端故障。

⑤ [配置] `makestep threshold limit` 是持久策略

常见配置：

```
makestep 1.0 3
```

表示在前若干次时钟更新中，如果修正量超过阈值，可通过 `step` 立即校正。阈值和次数必须按配置语义读取，不要把它误解为“每次偏差超过 1 秒都永久强制跳变”。

⑥ [配置] `rtcsync` 与 RTC 的关系

```
rtcsync
```

在 Linux 上，`rtcsync` 让内核在系统时钟处于同步状态时周期性更新 RTC。它不等同于每次运行 `hwclock --systohc`，也不能证明当前 RTC 与系统时钟已经完全一致。

⑦ [操作] 应用配置并分别验证服务状态

```
systemctl restart chronyd.service
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
systemctl status chronyd.service --no-pager -l
```

若题目同时要求当前启动和重启后自动启动：

```
systemctl enable --now chronyd.service
```

仍要分别验证 `active` 与 `enabled`，不能用一条 `status` 的视觉印象代替明确检查。

Cheatsheet

先备份并保留既有配置；`server` 单来源，`pool` 池来源；`iburst` 加快初始采样；`makestep` 是策略；`rtcsync` 让同步状态下的内核更新 RTC。

[操作专题] 解读 `chronyc activity` 与 `chronyc sources -v`

`sources -v` 是最常见的来源证据，但不能只看是否出现目标名称。必须同时看来源模式、选择标记、Reach、最近接收时间和最近样本。

① [操作] 先看来源总体活动状态

```
chronyc activity
```

它会汇总在线、离线、处于 burst、未解析等来源数量。若存在未解析来源，优先检查 DNS；若所有来源都离线，优先检查网络、配置和远端响应。

② [输出] 来源模式字符

在 `sources -v` 的第一列，常见模式包括：

- `^`：NTP server；
- `=`：peer；
- `#`：本地参考时钟。

本章主要训练客户端使用 `^` 来源，不展开 peer 和参考时钟部署。

③ [输出] 选择状态字符

标记	典型含义	下一步判断
<code>*</code>	当前选中的同步来源	再看 tracking 是否正常
<code>+</code>	可接受并参与组合的候选来源	正常候选，不必强制变成 <code>*</code>
<code>-</code>	可用，但未被选择参与组合	结合其他来源质量判断
<code>?</code>	不可选择：可能不可达、未同步或样本不足	查 DNS、网络、远端与样本
<code>x</code>	被判定为错误来源	调查来源冲突和质量，不要强制使用
<code>~</code>	时间变化过大	等待或调查抖动、网络和来源质量

来源模式和选择标记通常组合显示，例如 `^*`、`^+`、`^?`。

④ [输出] Stratum、Poll、Reach 与 LastRx

- `Stratum`：来源层级；数值不能单独决定优劣；
- `Poll`：以 2 的幂表示轮询间隔；
- `Reach`：最近八次传输的八进制可达寄存器；持续为 `0` 表示没有收到有效响应；
- `LastRx`：距最近一次有效样本的时间。

Reach 不是简单百分比，也不能在服务刚启动几秒时要求立即达到稳定值。

⑤ [输出] Last sample 的三组信息

最近样本通常显示：

```
adjusted_offset[measured_offset] +/- error_bound
```

括号外是经过处理的估计偏差，括号内是原始测量偏差，`+/-` 后是误差界。学习时重点是能识别符号、单位和数量级，不把任一值脱离来源状态直接当成最终业务结论。

⑥ [验证] 有目标名称不等于成功

最低可接受证据不是“输出中有 `time.example.com`”，而是：

名称已解析
→ Reach 开始积累有效响应
→ 来源不持续为 `^?` 或 `^x`
→ 至少存在 `^*` 或其他有效参考组合
→ tracking 不再 Not synchronised

Cheatsheet

activity 看总体；`^^` 是 server；`*`` 选中、`+`` 候选、`-`` 未选、`?`` 不可选、`x`` 错误源；Reach 持续 0 查往返链；最后必须进入 tracking。

[操作专题] 用 `chronyc tracking` 判断本机时钟是否真正受控

`sources` 以来源为中心；`tracking` 以本机系统时钟为中心。一个来源看起来可用时，还要确认本机是否已经选中参考并维持合理跟踪。

① [输出] Reference ID 与 Reference time

- `Reference ID`：当前参考来源的标识；可能显示地址或特定标识；
- `Reference time`：最近一次处理参考测量的时间。

若没有有效参考来源，Reference ID 可能为未同步状态。不要在讲义中背某个固定地址，因为真实环境不同。

② [输出] Stratum 与 Leap status

`Stratum` 表示本机在同步层级中的位置。`Leap status` 是重要的终态判断：正常跟踪时应处于正常状态；`Not synchronised` 明确说明尚未进入有效同步。

③ [输出] System time、Last offset 与 RMS offset

- `System time`：系统时钟相对 NTP 时间的当前估计差异；
- `Last offset`：最后一次时钟更新时的估计偏差；
- `RMS offset`：偏差的长期均方根统计。

它们回答不同时间窗口的问题，不能互相替代。正负号表示系统时钟相对参考时间的方向。

④ [输出] Frequency、Residual freq 与 Skew

`chronyd` 不只修正一次偏差，还估计本机时钟的频率误差：

- `Frequency`：系统时钟快慢的长期估计，单位通常为 ppm；
- `Residual freq`：当前仍未消除的频率差；

- **Skew**：频率估计的不确定性。

这些字段用于理解稳定性，不要求 RHCSA 学员根据单个数值手工调频。

⑤ [输出] Root delay、Root dispersion 与 Update interval

它们反映到参考时钟路径的延迟、累计误差和更新间隔。生产环境评估时间质量时有价值；考试最小验收仍以有效来源和正常 tracking 为核心。

⑥ [操作] 组合最小跟踪检查

```
chronyc sources -v
chronyc tracking
timedatectl
```

判定时避免只 grep 一个单词。建议人工确认：

```
存在有效选择来源
Reference time 合理更新
Leap status 不是 Not synchronised
System time/offset 没有持续发散
```

Cheatsheet

sources 回答“来源怎样”；tracking 回答“本机怎样”；重点看 Reference、Stratum、System/Last/RMS offset、Frequency/Skew、Update interval、Leap status。

[诊断专题] **chronyd active, 但指定来源持续显示 ^?**

^? 不是一个需要立即 step 的信号，而是“该来源当前不能参与选择”的症状。诊断要寻找最有区分度的下一条证据。

① [症状] 固定当前证据

```
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
chronyc activity
chronyc sources -v
chronyc tracking
```

记录目标来源是否未解析、Reach 是否为 0、LastRx 是否为空或过久，以及 tracking 是否 **Not synchronised**。

② [假设一] chronyd 没有读取预期配置

```
grep -nEv '^s*(#|$)' /etc/chrony.conf
systemctl status chronyd.service --no-pager -l
journalctl -u chronyd.service -b --no-pager
```

检查名称拼写、重复来源、编辑错文件、服务重启失败和配置解析错误。不要因为服务仍 active 就假设新配置已成功加载；重启失败时旧进程状态和退出信息都要看。

③ [假设二] 名称无法解析

```
getent ahosts time.example.com
chronyc activity
```

getent 使用系统名称服务链，比直接指定某个 DNS 工具更贴近应用解析结果。若失败，进入第 19 章的 NSS/DNS 诊断；本章只确认时间源名称是否能够得到地址。

④ [假设三] 网络路径或 UDP 123 往返失败

```
ip route get <resolved-address>
```

随后依据环境检查主机防火墙、上游 ACL/NAT 和远端服务。NTP 客户端通常从临时源端口向服务器 UDP 123 发送请求并等待响应；只开放本机入站 ntp 服务不是通用客户端修复。

普通 telnet 不能验证 UDP。nc -u 也很难证明对端提供了有效 NTP 响应，因此 chronyd 自己的 Reach、ntpdata（目标版本支持时）和抓包证据更有价值。

⑤ [假设四] 远端响应但样本尚不足

服务刚启动、DNS 刚恢复或来源刚上线时，^? 可能短暂存在。先确认 Reach/LastRx 是否开始变化，再等待合理的采样窗口。无限等待不是诊断；若 Reach 持续 0，应回到网络和远端响应。

⑥ [假设五] 来源本身未同步或被判异常

若能交换报文但来源仍不可选，检查远端 leap 状态、来源质量以及本机与其他来源的冲突。^x 表示 chronyd 已把它判为错误源，不能通过删除其他来源来“强迫”它变成正确源。

⑦ [最小修复与再验证]

修复必须对应已证实的故障层：

拼写错误 → 修正配置并重启 `chronyd`
DNS 错误 → 修复名称服务或使用题目允许的正确地址
路由/ACL 错误 → 修复具体路径或策略
远端不可用 → 使用题目授权的可用来源
样本不足 → 等待并观察 `Reach/LastRx`

再验证：

```
chronyc activity
chronyc sources -v
chronyc tracking
timedatectl
```

Cheatsheet

^^` 链路：配置加载 → DNS → 路由 → UDP 123 往返 → 远端状态 → 样本 → 选择 → tracking。修复哪一层，就重新验证哪一层及其下游。

[诊断专题] 大偏差时怎样选择 `slew`、`step` 与 `makestep`

时间偏差大并不自动授权跳变。首先判断当前偏差、系统所处阶段和业务对时间连续性的要求。

① [知识点] `slew` 渐进收敛

`slew` 通过临时改变系统时钟走速，使偏差逐渐消除。优点是墙上时钟通常不会突然向前或向后跳；缺点是大偏差需要较长时间才能收敛。

② [知识点] `step` 直接改变墙上时钟

`step` 立即把系统时钟移到目标附近。向前跳可能让超时提前，向后跳可能产生时间重复。日志排序、数据库事务、缓存过期、证书和调度器都可能受影响。

③ [配置] 启动阶段的 `makestep`

```
makestep 1.0 3
```

其目的通常是允许启动早期在有限次数内修正大偏差。它比在系统长期运行后无条件 `step` 更可控，但仍需理解阈值和次数。

④ [运行时操作] `chronyc makestep`

```
chronyc makestep
```

这是对正在运行的 `chronyd` 发出的控制命令，可能立即造成时间跳变。它不是持久配置，也不是 `^?` 的修复命令。执行前至少要确认：

```
当前有效参考源是谁
当前偏差是多少
是否允许时间跳变
关键应用是否已评估或停机
执行后怎样验证
```

⑤ [边界] 先修来源，再处理偏差

没有有效参考源时，`step` 没有可信目标。来源持续 `^?` 时先修复来源链。若系统完全离线且题目要求手工设置时间，应明确真源、授权和后续恢复网络同步的步骤。

⑥ [再验证] 校正后不是只看 `date`

```
date
date -u
chronyc sources -v
chronyc tracking
timedatectl
```

还要检查关键应用、日志时间和计划任务影响。应用专项验证不在本章完整展开，但必须写入变更记录。

Cheatsheet

`slew` 保持连续、渐进收敛；`step` 立即跳变；配置 ``makestep`` 是有限策略，``chronyc makestep`` 是运行时动作；没有有效来源时先修来源。

[知识专题] 时间证据的可信边界与工作迁移

时间同步成功后，系统能够更可靠地记录新事件，但它不会自动修复过去已经写错的时间戳。跨主机调查也不能只看到“都启用了 `chronyd`”就认为时间轴一致。

① [边界] 错误时区与错误系统时钟的影响不同

错误时区通常不改变事件的 UTC 先后，但会误导人类阅读和本地时间范围查询。错误系统时钟则会真正改变写入的时间戳，破坏跨主机排序。

② [边界] 后续同步不会改写历史日志

某事件在错误系统时间下写入后，`chronyd` 后续恢复同步不会把旧日志时间自动纠正。调查者要结合启动轮次、消息内容、外部证据和偏差窗口重建时间线。完整 `journal` 技巧归第 13 章。

③ [边界] active 不代表集群时间一致

多台主机都显示 `chronyd active`，仍可能分别跟踪不同来源、处于未同步状态或有不同偏差。跨主机验收至少要比对来源、Reference time、Leap status 和偏差，而不是只比较服务状态。

④ [工作迁移] 变更前后保存证据矩阵

推荐记录：

```
本地时间 / UTC / 时区
RTC 模式与读数
chronyd active / enabled
配置的 server / pool
DNS 解析结果
activity / sources / tracking
是否执行过 step
应用影响与观察窗口
```

⑤ [工作迁移] 自动化应等待条件，不只等待服务启动

在脚本或自动化中，`systemctl start chronyd` 返回成功仅表示启动请求完成。需要时间正确后再启动关键业务时，应使用 `chronyc waitsync` 等条件式等待，并根据目标版本定义可接受尝试次数和偏差；本章只作为扩展，不把它替代人工理解。

Cheatsheet

后续同步只改善未来，不改写历史；跨主机比较 sources/tracking，不只看 active；变更记录要包括是否发生 step。

[经典任务] 配置指定时区和时间源，并诊断 active 但持续 ^?

环境

你管理一台 RHEL 9 主机。当前没有可由本讲义控制的 live VM，以下任务用于静态训练；在真实练习机上必须读取实际输出。

已知状态：

- 当前时区不是 `Asia/Shanghai`；
- `chronyd.service` 当前为 active，且配置为开机启动；
- `/etc/chrony.conf` 已有其他有效指令，禁止清空或整文件覆盖；
- 题目要求加入指定来源：

```
server time.example.com iburst
```

- 修改后，该来源在 `chronyc sources -v` 中持续显示 `^?`；

- 尚未确定是配置加载、DNS、网络、远端响应还是样本不足。

目标终态

1. 时区持久设置为 `Asia/Shanghai`；
2. 能解释本地时间、UTC 和 RTC 的关系；
3. RTC 模式被检查，未在无明确需求时改成本地时间；
4. 指定来源配置存在且不重复，既有有效配置被保留；
5. `chronyd` 当前运行且重启后自动启动；
6. 指定名称能够通过系统名称服务解析；
7. 指定来源取得有效测量，不再持续为 `^?` 或 `^x`；
8. 至少存在一个有效参考来源；
9. `chronyc tracking` 不显示 `Not synchronised`；
10. 未经授权，不通过手工改时或强制 `step` 掩盖来源故障。

限制条件

- 不关闭 SELinux 或整个防火墙作为默认答案；
- 不删除其他有效时间源来强迫错误源被选中；
- 不把“服务 active”“命令返回 0”“输出中有源名称”当成最终完成；
- 不编造真实 IP、Reach、offset、Reference ID 或标准输出；
- 不把固定毫秒阈值当作所有环境的通用标准。

验收证据

层次	推荐证据	验收意图
时间显示	<code>date</code> 、 <code>date -u</code> 、 <code>timedatectl</code>	时区与 UTC 关系正确
RTC	<code>timedatectl</code> 、 <code>hwclock --show</code>	RTC 模式已检查
配置	检查 <code>/etc/chrony.conf</code>	目标源存在且保留既有配置
服务当前/持久	<code>is-active</code> 、 <code>is-enabled</code>	两个状态分别成立
解析	<code>getent ahosts time.example.com</code>	系统解析链可用
来源	<code>activity</code> 、 <code>sources -v</code>	来源可达并参与选择
跟踪	<code>tracking</code>	本机进入有效跟踪
安全边界	变更记录	未无调查执行 <code>step</code>

[参考解答] 从基线、最小修改到分层验收

① 调查并保存修改前基线

```
date '+local=%F %T %z %Z'
date -u '+utc=%F %T %z %Z'
timedatectl
hwclock --show
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
grep -nEv '^s*(#|$)' /etc/chrony.conf
chronyc activity
chronyc sources -v
chronyc tracking
```

调查阶段不尝试 step。若 `hwclock` 在虚拟机中不可用，记录错误并继续调查 system clock 与 chrony，不把它误判为 NTP 来源故障。

② 设置时区并验证持久状态

```
timedatectl list-timezones | grep '^Asia/Shanghai$'
timedatectl set-timezone Asia/Shanghai

timedatectl
date '+local=%F %T %z %Z'
date -u '+utc=%F %T %z %Z'
readlink -f /etc/localtime
```

这里验证的是显示和持久时区，不是 NTP 同步。

③ 备份并最小编辑 chrony 配置

```
cp -a /etc/chrony.conf /etc/chrony.conf.pre-rhcsa14
grep -nF 'time.example.com' /etc/chrony.conf
```

若不存在目标行，在不删除其他有效指令的前提下加入：

```
server time.example.com iburst
```

再次检查避免重复：

```
grep -nF 'time.example.com' /etc/chrony.conf
```


④ 应用配置并验证服务层

```
systemctl restart chronyd.service
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
systemctl status chronyd.service --no-pager -l
journalctl -u chronyd.service -b --no-pager
```

若题目发现服务未启用，再执行：

```
systemctl enable --now chronyd.service
```

不要因为 restart 返回成功就跳过日志和状态检查。

⑤ 针对 ^? 选择下一条证据

先查系统解析链：

```
getent ahosts time.example.com
chronyc activity
```

若名称能够解析，选择其中实际地址检查路由：

```
ip route get <resolved-address>
```

然后依据环境检查具体防火墙、ACL、NAT 和远端 NTP 服务。不得用“关闭所有防火墙”替代定位。若有限权和工具，可用抓包确认请求与响应，但不在答案中虚构抓包结果。

⑥ 判断是持续故障还是采样尚未完成

```
chronyc sources -v
```

观察：

- Reach 是否持续为 0；
- LastRx 是否开始更新；
- 目标源是否仍持续 ^?；
- 是否出现 ^x 或 ^~；
- 其他来源是否正常。

若 Reach 开始积累，可等待合理采样窗口后再查；若持续为 0，继续调查往返链和远端响应。

⑦ 完成来源与跟踪验收

```
chronyc activity
chronyc sources -v
chronyc tracking
timedatectl
```

验收逻辑：

```
目标名称已解析
→ 来源取得有效响应和样本
→ 不持续为 ^? 或 ^x
→ 存在有效选中/组合来源
→ tracking 不为 Not synchronised
→ 时区和服务持久状态仍正确
```

题目要求“使用指定来源”时，应确认该来源本身进入有效状态；若还有多个来源，不能武断要求它一定显示 `^*`，除非题目明确要求唯一来源或当前选择逻辑应当选中它。

⑧ 何时才考虑 `makestep`

只有在以下事实明确后才评估：

```
已有可信参考源
当前偏差确实很大
业务允许时间跳变
题目或变更授权允许
```

运行时命令：

```
chronyc makestep
```

执行后重新检查 `date`、`sources -v`、`tracking`，并检查日志、认证、数据库和计划任务影响。经典任务未授权 `step`，因此默认答案不执行它。

⑨ 典型错误

错误	为什么不成立
看到 active 就结束	只证明守护进程层
看到源名称就结束	可能仍是 <code>^?</code> 、Reach 0 或未选中
<code>ping</code> 不通就认定 NTP 不通	ICMP 与 UDP 123 是不同协议链
用 <code>telnet host 123</code> 验证	telnet 验证 TCP，NTP 通常使用 UDP

错误	为什么不成立
直接 <code>chronyc makestep</code>	不修复 DNS、网络或错误来源
删除所有其他来源	可能降低冗余并掩盖错误源
关闭整个防火墙	破坏性大，且未定位具体规则
把 RTC 改成本地时间	通常增加 DST 和多系统风险

[本章收束] 把时间管理变成证据矩阵

本章的核心不是“让命令不报错”，而是把时间链拆成可独立验证的对象：

- UTC / 本地显示 / 时区
 - `system clock`
 - RTC
 - `chronyd` 当前与持久状态
 - 配置的 `server / pool`
 - DNS 与 UDP 123 往返
 - `sources` 测量与选择
 - `tracking` 跟踪状态
 - `step/slew` 安全边界

面对时间异常时，先问“哪个对象错误”，再选择命令。面对 `^?`，按照配置、解析、网络、远端、样本、选择、跟踪推进。面对大偏差，先建立可信来源和影响评估，再决定是否允许 `step`。

主要判断表

看到的证据	可以证明	不能证明
<code>chronyd.service</code> 为 active	守护进程当前运行	已有有效来源或已经同步
<code>sources -v</code> 中出现来源名称	<code>chronyd</code> 知道该来源	来源可达、可选或被选中
来源显示 <code>^*</code> 或 <code>^+</code>	来源已被选中或参与组合	偏差已经满足具体业务阈值
<code>tracking</code> 的 Leap status 为 <code>Normal</code>	本机正处于有效跟踪状态	其他主机也与本机一致
<code>date</code> 显示看起来正确	当前显示结果合理	来源可信、RTC 正确或历史日志已修复

最终 Cheatsheet

```
date '+%F %T %z %Z'; date -u '+%F %T %z %Z'
timedatectl
timedatectl set-timezone Asia/Shanghai
hwclock --show
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
chronyc activity
chronyc sources -v
chronyc tracking
getent ahosts time.example.com
ip route get <resolved-address>
journalctl -u chronyd.service -b --no-pager
```

向下一章交接

本章已经建立“时区改变本地日历解释、step 造成墙上时钟跳变”的边界。第 15 章将在此基础上讨论 `at`、`cron` 与 `systemd timer` 的持久配置、执行环境和实际触发行为。